

Fast GPU Material-Removal Simulation for 3-Axis Milling

Fused swept-segment subtraction on a single-axis dixel field, with tube pruning

autocam — technical paper

7 August 2026

Abstract

Simulating material removal (the “carving” of a stock by a rotating tool along a G-code toolpath) is the forward model at the heart of learned and search-based toolpath generation: it must be **fast**, because it is evaluated inside an optimization loop, and **accurate**, because its output is the training signal. We present the GPU carving pipeline of **autocam**, a 3-axis milling simulator built on OpenGL compute shaders. Stock and tool are stored as a **single-axis dixel field** — per-column sorted lists of Z-transitions — on which a Boolean difference is an embarrassingly parallel per-column merge. On top of a **fused swept-segment subtraction** (one Boolean difference per linear toolpath segment, whose swept-tool column is computed on the fly as a Z-envelope and subtracted in place, without ever materializing the swept volume), we introduce **tube pruning**: because the GPU dispatch covers the axis-aligned bounding box of a segment’s endpoints — far larger than the actual swept band on diagonal moves and in-air rapids — we skip the per-column write-back for columns the tool never touches. This attacks *memory-write traffic*, which we show is the true bottleneck, rather than compute. The optimization is bit-exact (byte-identical output) and delivers, on an Intel Lunar Lake iGPU, $\sim 2.6\text{--}2.9\times$ on a 45° raster, $\sim 5\times$ on long diagonal cuts and $\sim 7.6\times$ on in-air rapids, while leaving axis-aligned programs unchanged. We also report two **documented negative results** — a range-query envelope acceleration and an oriented tiled dispatch — that together localize the bottleneck precisely: on this pipeline the lever that pays is the one that removes memory traffic, not the ones that remove compute or launched threads.

Keywords: NC machining simulation, material removal, dixel, swept volume, GPU compute, memory bandwidth, 3-axis milling.

1. Introduction

1.1 Motivation

CNC toolpath data is scarce in public datasets because it is proprietary. Our long-term goal is to **synthesize training data** for a learned toolpath generator: given a stock block and a cutting tool, produce the G-code that machines a target part. Learning this map requires many (G-code $\rightarrow$ resulting geometry) pairs, which in turn requires a **fast, accurate forward model** — a simulator that, given a toolpath, a tool and a stock, computes the machined geometry. That forward model sits inside an optimization loop (a genetic algorithm searches toolpaths and scores each against a target), so it may be evaluated **millions of times**. Raw simulation speed at fixed accuracy is therefore the central engineering requirement. We restrict the scope to **3-axis milling** (tool translation only; the tool axis is fixed to Z).

1.2 Related work

Column-based material-removal simulation begins with the **dexel model** of Van Hook (1986) [1], which renders NC milling in real time as an image-space Boolean subtraction: the workpiece is a grid of Z-direction “depth elements” (dexels), and the tool lowers their heights. The model was extended to **multi-/tri-dexel** representations (three orthogonal dexel bundles) to improve sampling on steep walls: Müller & Surmann (2003) [2] present online sculpting on multi-dexel volumes, and GPU variants were developed by Tukora & Szalay (2012) [3] and Inui, Kobayashi & Umezue (2019) [4]. In parallel, the solid-modeling community proposed **sparse, per-axis implicit** representations with GPU Booleans, notably **Layered Depth-Normal Images** (LDNI, Wang, Leung & Chen 2010 [5]). Swept-volume computation has its own line, from the sweep-envelope differential equation of Blackmore, Leu & Wang (1997) [6] to the survey of Abdel-Malek et al. (2006) [7]. Surface reconstruction uses **Marching Cubes** (Lorensen & Cline 1987 [8]; on the GPU, Dyken et al. 2008 [9]).

Two observations frame our contribution. First, **subtracting a tool’s swept volume per move from a dexel model is not new** — it is exactly what [2,3,4] do. Second, once the obvious per-step overheads are removed, such a simulator is limited not by geometry but by **memory bandwidth on the working-set buffer**. Our contribution is therefore not a new representation, but (i) an efficient *materialization-free, single-axis* GPU realization of swept-segment subtraction, and (ii) **tube pruning**, an optimization that targets memory-write traffic — the actual bottleneck — which the prior GPU-dexel line does not address.

1.3 Contributions

- A **fused swept-segment subtraction** on a single-axis (Z) dixel field: one Boolean difference per linear toolpath segment, whose swept-tool column is computed on the fly as a Z-envelope and subtracted in place, with no intermediate swept-volume buffer (§3.3).
 - **Tube pruning** (§3.6): skipping the per-column write-back for columns outside the swept band (diagonal over-dispatch) or outside the stock's Z-range (in-air rapids). Bit-exact; 2.6–7.6× in its regime (§6).
 - A **bit-exact validation methodology** (order-independence + Chebyshev sampling + a residual-volume invariant) and a **runtime A/B toggle** so baseline and optimized run from one binary (§5).
 - Two **documented negative results** (range-query envelopes; oriented tiled dispatch) that localize the bottleneck to memory-write bandwidth (§7).
-

2. Volume representation: single-axis dixel field

Both stock and tool are stored not as a dense 3D occupancy grid but as a **single-axis dixel field along Z**. For a grid of $W \times H \times D$ voxels, each (x, y) **column** of D voxels is encoded as the **sorted list of Z heights at which occupancy flips** (empty→solid or solid→empty). Because milling parts are mostly solid with a few bounding surfaces, a column typically has only 2–4 transitions; a plain stock column is just $[0, T]$.

Z-transition column representation (single-axis dixel)

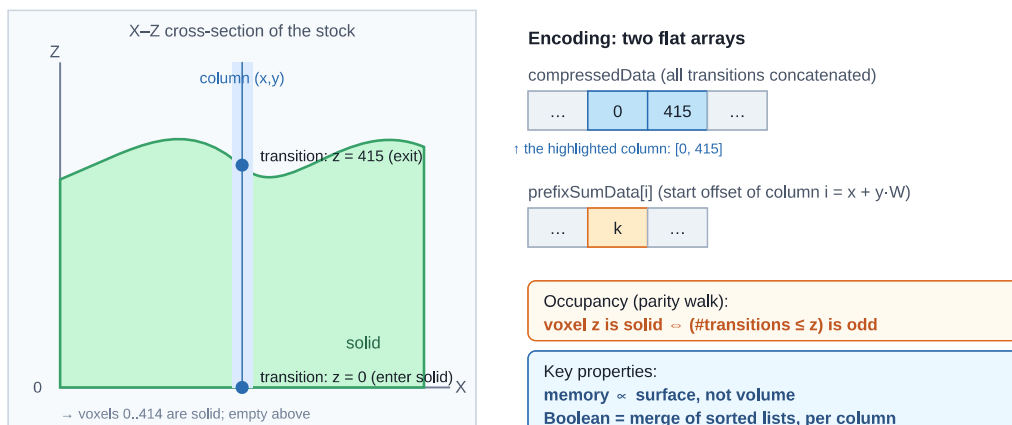


Figure 1. The single-axis dixel (Z-transition column) representation. Each column stores the sorted heights where occupancy flips; two flat arrays encode the whole

object, and occupancy is recovered by a parity walk. Storage is proportional to surface complexity, not volume, and any Boolean operation is a merge of sorted lists, independent per column.

The whole object (struct VoxelObject) is two flat arrays:

- `compressedData` — all columns' transition lists, sorted ascending, concatenated.
- `prefixSumData` — for each column $i = x + y \cdot W$, the start offset of its list in `compressedData`.

Occupancy at (x, y, z) is recovered by a **parity walk**: voxel z is solid iff an *odd* number of transitions are $\leq z$. Three properties are exploited throughout: **(i) compactness** (storage grows with surface area, not volume: a $1000 \times 1000 \times 500$ stock is $\approx 2 \cdot 10^6$ transitions, a few MB, versus 500 M dense voxels); **(ii) Boolean = per-column merge**, independent across the $W \cdot H$ columns, hence massively parallel on the GPU; **(iii) a free bounding interval** — a column's lowest and highest transitions are exactly its solid extent [bottom, top], a single interval for tools whose columns are single Z-intervals (§4.5).

3. From naive stamping to the final solution

We build the method as a sequence of steps, each validated bit-exactly (§4.6). A running example is used throughout: a stock $1000 \times 1000 \times 500$; a **ball-end mill** (a cylinder with a hemispherical tip) of radius R ; and a linear move of the tool centre from p_0 to p_1 .

3.1 Naive stamping (baseline)

The naive simulator advances the tool in small fixed **jog steps** (e.g. $s = 2$ voxels) and **stamps the whole tool** — one Boolean-difference dispatch — at every step. For a segment of length L this is $\approx L/s$ subtractions; consecutive stamps re-process columns already carved, and, when $s > 1$, leave **scallops** between samples.

Stamping (baseline) vs Swept subtraction

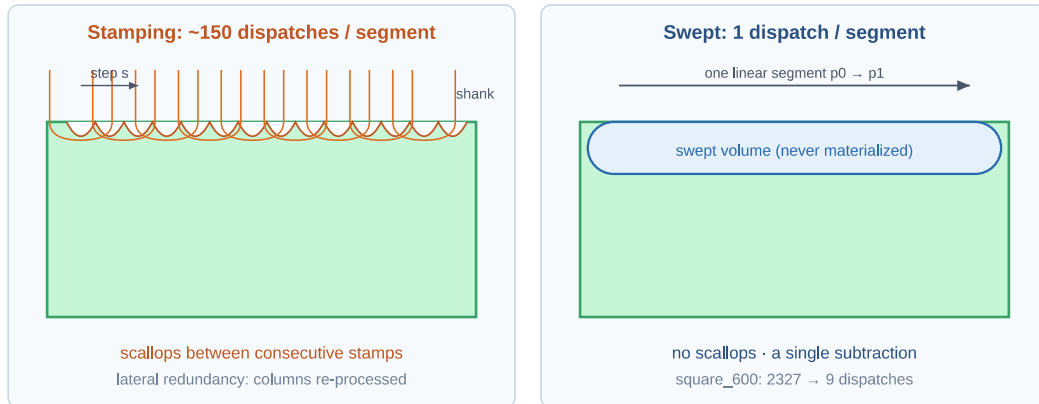


Figure 2. Stamping versus swept subtraction. Left: many overlapping stamps along the path, with residual scallops and lateral redundancy (the tool is a ball-end mill — hemisphere tip plus shank). Right: a single subtraction of the swept volume, with a clean floor. On the square_600 benchmark the dispatches drop from 2327 to 9.

3.2 Swept-segment subtraction

Instead of L/s stamps we perform **one subtraction per linear segment**, of the tool's **swept volume** (the Minkowski sum tool $\oplus p_0p_1$). Set difference is order-independent and distributes over unions, so subtracting the swept volume once equals stamping at every sampled position along the segment (§4.6). Effect: on square_600 the dispatches drop from 2327 to 9.

3.3 On-the-fly Z-envelope, fused in one kernel (core contribution)

The subtlety is that “subtract the swept volume” would seem to require **building** it in a separate buffer and then subtracting it. We never do. Because the tool's XY columns are single solid intervals, moving it by ≤ 1 voxel per sub-step makes the intervals a given stock column “sees” **overlap**; the union of overlapping intervals is still one interval [min bottom, max top] — the **Z-envelope**. A single fused compute shader computes this envelope on the fly, per column, and subtracts it in place in the same pass.

Per-column Z-envelope, computed on the fly (the core of the method)

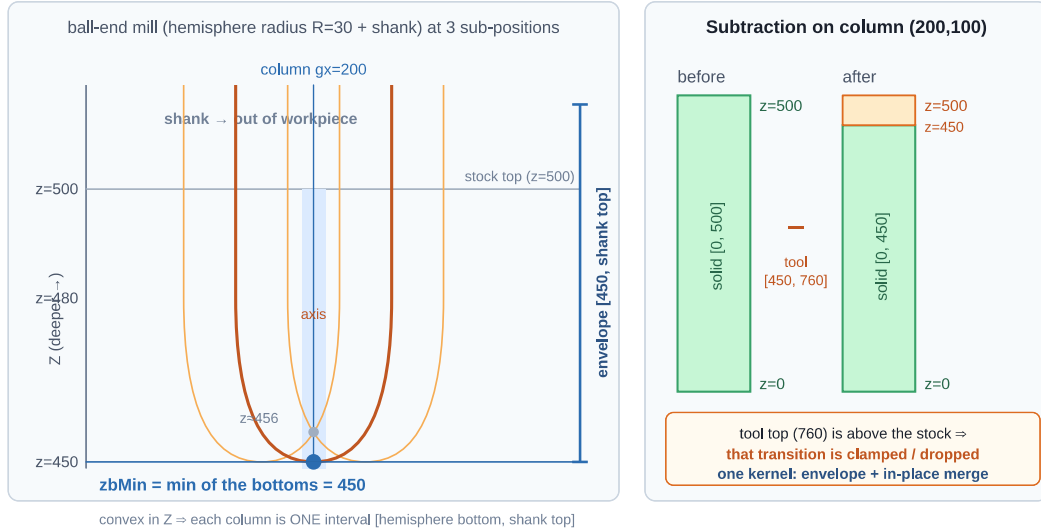


Figure 3. The per-column Z-envelope. Left: the ball-end mill (hemisphere tip + shank) at three sub-positions; the shank rises out of the workpiece. For a given column the deepest bottom is reached when the tool axis passes over it; the envelope is $[zbMin, \text{shank top}]$. Right: its subtraction from the column, done in place in the same kernel — the tool's top transition, above the stock, is clamped.

Worked example. A stock column starts as $[0, 500]$. As the tool centre slides in X , the column is covered for a contiguous sub-range of positions; for each, the hemisphere bottom at that lateral offset is computed. The envelope takes the minimum bottom (deepest, on-axis) and the maximum top (the shank top, above the stock). Subtracting $[450, \text{shankTop}]$ from $[0, 500]$ — with the shank-top transition clamped to the stock range — yields $[0, 450]$: exactly what the 150 stamps would produce, via a short loop and one merge, with **no buffer for the swept volume**.

3.4 Sub-step bounding

A column is covered by the tool only for a contiguous range of sub-steps $[k_0, k_1]$, obtainable in closed form from the tool centre's linear motion. Iterating only that range (with ± 1 safety margins) drops the per-column work from *segment length* to $\approx$ *tool extent* on axis-aligned moves.

3.5 GPU compaction for read-back

After carving, the result must be returned in the compact column format. Reading back the unpacked working buffer would be $W \cdot H \cdot 32 \cdot 4 \text{ B} \approx 128 \text{ MB}$. Instead

we compact **on the GPU** first (§4.2), so only ~8 MB of dense transitions cross the bus; read-back drops from ~72 ms to ~8 ms.

3.6 Tube pruning (the new optimization)

The dispatch (§4.3) covers the **axis-aligned bounding box (AABB)** of the two **tool-centre endpoints, dilated by the tool footprint**. For an axis-aligned move that AABB already equals the swept band, but for a **diagonal** move of length L and tool footprint D it is $\approx (0.7L + D)^2$ columns while the tool sweeps only $\approx L \cdot D$ — and every over-covered column still rewrote all 32 slots for a no-op, because the shader had no early-out before its write-back. In-air rapid (G0) moves are the same: the tool is above the stock, so every column is a no-op that still rewrites 32 slots.

Tube pruning: dispatch AABB vs. the swept band

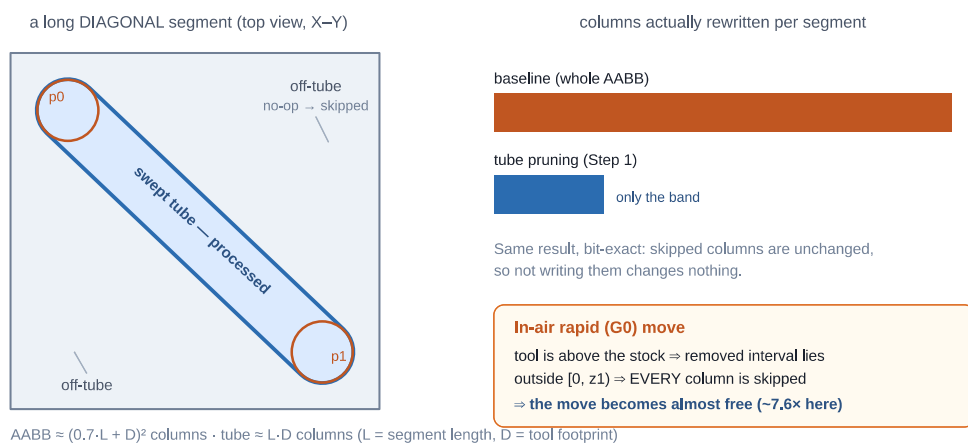


Figure 7. Tube pruning. On a long diagonal segment the dispatch AABB (dashed) is far larger than the swept band it contains; the off-band columns are no-ops that were still being rewritten. Skipping their write-back leaves them untouched (bit-exact) and removes that memory traffic. In-air rapids are the extreme case: the removed interval is entirely outside the stock's Z-range, so every column is skipped.

The fix is a single early-out in the shader (§4.4): before the merge and write-back, return when the tool never covers the column (`hasMat == false`) or the removed interval lies entirely outside the stock's Z-range (`zbMin  $\geq$  z1 || ztMax  $\leq$  0`). The column is left untouched — correct, because it is unchanged — and its 128 bytes of write traffic vanish. This is what makes over-dispatched diagonals and in-air rapids cheap; axis-aligned dispatches are fully covered and unaffected.

Segment-level pruning. The same idea applies one level up, on the host: when the tool's Z-extent over a *whole* segment cannot reach the stock's $[0, z_1)$ range — the common case for an in-air G0 rapid (which maps below the stock under the inverted-Z convention) — the segment removes nothing and its **entire dispatch is skipped**, not merely pruned per column. Rapids become essentially free (a synthetic all-rapid program: $\sim 2900\times$) and every program with repositioning moves speeds up; it is host-side, bit-exact, and gated by the same toggle.

4. Algorithm and GPU implementation

4.1 A short GPU-compute primer

The pipeline uses **compute shaders** — programs that run on the GPU outside the graphics pipeline. A program is launched by a **dispatch** that creates a grid of **workgroups**; each workgroup contains a fixed block of **threads** (here $\text{local_size} = 8 \times 8 \times 1$, so 64 threads) that execute the shader in parallel. Each thread reads its `gl_GlobalInvocationID`, a 3-D index identifying it within the whole grid. Data lives in **Shader Storage Buffer Objects (SSBOs)** — large GPU-resident arrays the shader reads and writes. Ordering between dependent dispatches is enforced by a **memory barrier**. In our carver **one thread processes one workpiece column**; this is what makes the per-column-merge property of the dixel representation (§2) map directly onto the hardware.

4.2 Data layout: the flat working buffer

During a carving session the stock is held GPU-resident in a fixed-stride **flat** buffer, `obj1_flat`, of $W \cdot H \cdot \text{MAX_TRANSITIONS}$ uints ($\text{MAX_TRANSITIONS} = 32$), plus a per-column count buffer `obj1_dataNum`. Column (x, y) owns the 32-slot slice `obj1_flat[idx·32 ... idx·32+32)`, $\text{idx} = x + y \cdot W$. Subtractions write results **in place** into the same slots, so no per-operation reallocation is needed. This buffer is $1000 \cdot 1000 \cdot 32 \cdot 4 \text{ B} = 128 \text{ MB}$ for the reference stock — the working set whose bandwidth dominates the cost (§7). The tool is uploaded once as a compact (`compressedData`, `prefixSumData`) pair and never mutates.

4.3 Dispatch: mapping threads onto the XY grid

For each segment the host computes the swept bounding box — the AABB of the two tool-centre endpoints, dilated by the tool half-footprint ($w/2$, $h/2$) and clamped to the grid — giving an origin (`baseX`, `baseY`) and a size (`bboxW`, `bboxH`). It then dispatches $\lceil \text{bboxW}/8 \rceil \times \lceil \text{bboxH}/8 \rceil$ workgroups. Inside the shader each thread

recovers its column as $gx = \text{baseX} + \text{gl_GlobalInvocationID.x}$, $gy = \text{baseY} + \text{gl_GlobalInvocationID.y}$, and returns immediately if $gx \geq W$ or $gy \geq H$ (the ragged edge of the last workgroups).

Mapping compute threads onto the workpiece column grid

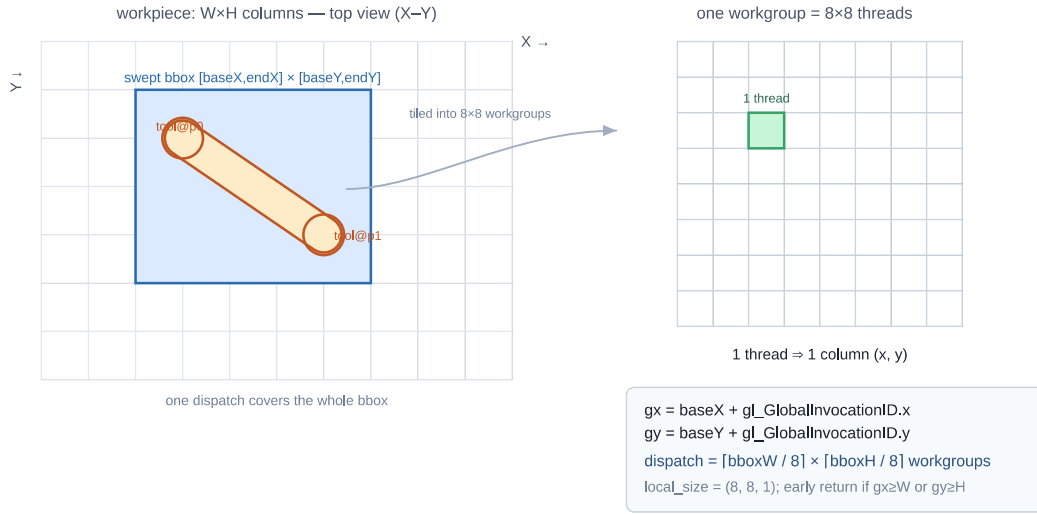


Figure 5. Mapping compute threads onto the workpiece column grid. The host restricts the dispatch to the swept bounding box; the GPU tiles it into 8×8 workgroups, and each thread handles one column (x,y) recovered from its global invocation ID. The number of workgroups is $\lceil \text{bboxW} / 8 \rceil \times \lceil \text{bboxH} / 8 \rceil$.

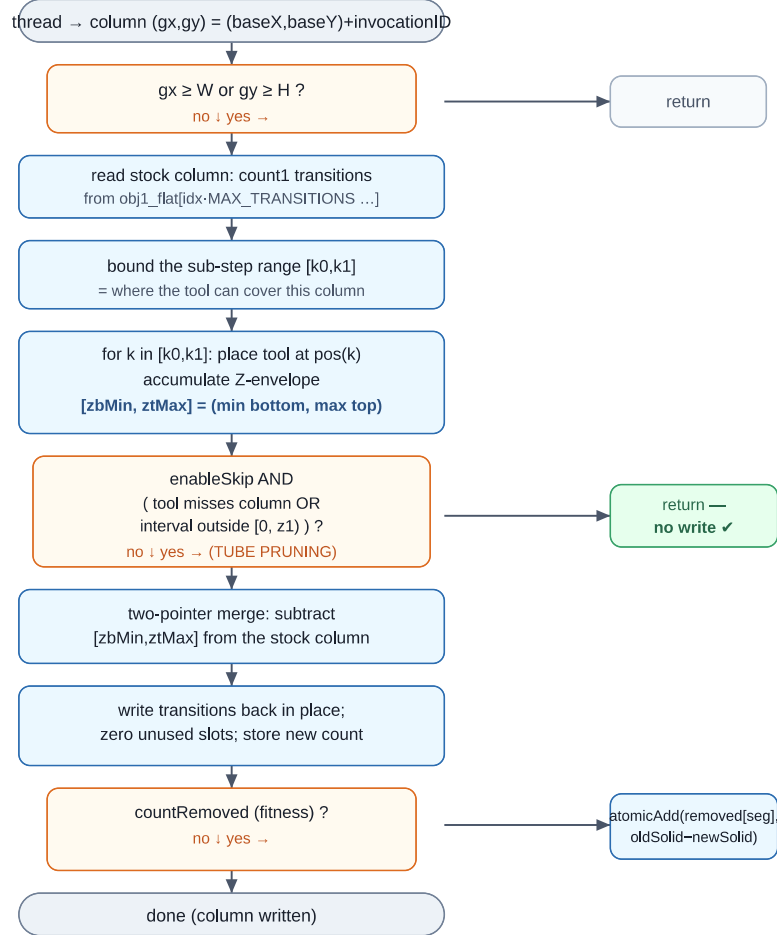
4.4 The subtract_swept shader, step by step

Each thread runs the following (Figure 6). It is deliberately branch-light and reads the tool from cached buffers while touching the large obj1_flat buffer exactly once per column.

1. **Identify the column** and bounds-check it against W , H .
2. **Read the stock column** — its transition count count1 from obj1_dataNum , its transitions from obj1_flat .
3. **Bound the sub-step range** $[k0, k1]$ — the sub-positions of the tool centre for which the tool could cover this column (from the linear motion $\text{tr} \approx \text{start} + k \cdot \Delta / K$).
4. **Accumulate the Z-envelope** — loop k over $[k0, k1]$; for each, find the tool's column under (gx, gy) and update $\text{zbMin} = \min(\text{zbMin}, \text{bottom})$, $\text{ztMax} = \max(\text{ztMax}, \text{top})$.

5. **Tube-pruning early-out** — if enableSkip and (the tool never covered the column, or the interval [zbMin, ztMax] lies entirely outside [0, z1]), return **without writing** (§3.6).
6. **Subtract** — a two-pointer merge of the stock column with the single interval [zbMin, ztMax], emitting a transition whenever the *result* occupancy (stockInside AND NOT toolInside) changes.
7. **Write back** the result transitions in place, zero the unused slots, and store the new count.
8. **Optional counter** — when material-removal tracking is on (used by the fitness evaluator), atomicAdd(removed[segment], oldSolid – newSolid).

subtract_swept.comp — per-thread flow (one column)



Blue = compute step · Orange = branch · Green "no write" = the tube-pruning early-out (Step 1).
Every column of the swept bbox runs this independently, one GPU thread each.

Figure 6. Per-thread control flow of *subtract_swept.comp*. Blue boxes are compute steps, orange are branches; the green “no write” return is the tube-pruning early-

out. Every column of the swept bounding box runs this independently, one GPU thread each.

Two design points matter for performance. The **merge is in place** and touches the 32-slot column once; and the **envelope reuses the column's lowest/highest transition** rather than an intermediate swept solid. The tube-pruning early-out (step 5) is the only change of §3.6, and it removes the write of step 7 — the expensive part — for no-op columns.

4.5 Tool geometry

A ball-end mill is not a sphere but a **cylinder with a hemispherical tip**; its shank rises above the stock and, in normal 3-axis milling, is out of the workpiece at every instant. This is compatible with the single-interval property: the hemisphere-plus-shank solid is convex in Z, so each of its XY columns is one interval [hemisphere bottom, shank top]. The shank top is a genuine transition in the tool's representation, but because it lies above the stock it is clamped away by the $[0, z_1)$ bound during subtraction. The method is exact for tools whose columns are single Z-intervals (ball, flat, conical, bull-nose) and over-removes gracefully for non-convex tools — the same behaviour as the discrete stamper.

4.6 Correctness: bit-exact against the stamper

The swept result equals discrete stamping **to the byte**, for two reasons. **Order-independence**: set difference distributes over unions and is order-independent, so $A \setminus (B_1 \cup \dots \cup B_n) = (\dots((A \setminus B_1) \setminus B_2) \dots \setminus B_n)$; the swept volume is the union of the tool at the sampled positions, so subtracting it once equals stamping them all.

Chebyshev sampling: sub-positions are sampled at step $K = \max(|\Delta x|, |\Delta y|, |\Delta z|)$ (the Chebyshev/ L_∞ length), guaranteeing ≤ 1 -voxel motion per sub-step, so the envelope touches every integer position the stamper would. Tube pruning only *skips writes to unchanged columns*, so it is bit-exact by construction. The invariant used to check this is the **residual solid volume** — the sum, over all columns and their solid intervals, of (top – bottom) — a deterministic, render-independent quantity.

5. Numerical experiments

5.1 Hardware, software, and the A/B toggle

Measurements are on an **Intel Lunar Lake integrated GPU** (Mesa iris driver), OpenGL 4.6 compute, which autocam reaches via an **EGL/Wayland** context.

(Note: the X11 `glxinfo` utility reports `llvmpipe` on this system, but that is the GLX path, not the context autocam uses; the real driver is confirmed from the EGL log.) The tube-pruning early-out is gated by an `enableSkip` uniform driven by the environment variable `AUTOCAM_SWEPT_SKIP` (default on), so **baseline and optimized run from the same binary** — essential for a fair A/B. All runs are headless (`--no-view`).

5.2 Benchmarks and the G-code generator

Because the optimization targets segments whose length L exceeds the tool footprint D , the benchmark set must exercise that regime — long, non-axis-aligned segments and air travel — which the project's original programs (a square perimeter, star pocket, rectangular pocket) do **not** when run with the wide 256-voxel `hemispheric_mill_10`. We therefore use:

- `square_600` with `hemispheric_mill_10` (256 wide) — an **axis-aligned control** (AABB already equals the tube; expected $\sim 1\times$).
- `bench_complex` with `hemispheric_mill_3` (32 wide) — a **45° raster** with 13 long diagonal cutting passes interleaved with long diagonal air repositions, generated programmatically.
- Two stress micro-benchmarks with the 32-wide tool: `diag_cut` (repeated long diagonal cuts) and `air_moves` (repeated long diagonal G0 rapids).

The 45° raster is emitted by a small parametric generator that walks lanes in a rotated basis (cutting direction $\hat{u} = (1,1)/\sqrt{2}$, step direction $\hat{v} = (1,-1)/\sqrt{2}$), converting each (u,v) to (x,y) ; each lane is a long diagonal G1 cut followed by a retract and a long diagonal G0 reposition to the next lane. This mirrors a real finishing raster while maximizing both effects the optimization targets.

5.3 Validation

For every configuration we save the carved `.bin` and confirm it is **byte-identical** across baseline, skip, and tube modes, and that its residual solid volume matches the project's reference numbers (`square_600` = 281 165 072, `star_pocket` = 462 722 118). The residual volume is computed offline by an independent pure-Python reader (no dependency on the GPU path).

5.4 Methodology

Each configuration is run several times; the **first run is discarded** as compute-pipeline warm-up and the **minimum** net-carving time is reported. Crucially, the modes are **interleaved per round** (one run of each mode per round) rather than

measured in separate batches, so that GPU clock/thermal drift cancels in the ratio — without this, the never-taken branch of the axis-aligned control spuriously appears 30% slower. Timing uses the simulator’s internal “net carving” timer (a `glFinish` around the per-segment loop, excluding read-back).

6. Results

Table 1 gives net carving time (ms) for the three modes and the resulting speedups; Figure 8 shows the same, normalized to baseline.

Table 1 — Net carving time (ms), Intel LNL / iris, interleaved min-of-N, all bit-exact.

benchmark (tool)	baseline	skip (Step 1)	tube (Step 2)	skip speedup	tube speedup
square_600 (mill_10, control)	23.7	24.5	28.0	0.97×	0.85×
bench_com plex (mill_3)	42.7	14.8	23.6	2.89×	1.81×
diag_cut (mill_3)	31.7	6.2	12.4	5.08×	2.56×
air_moves (mill_3)	29.6	3.9	7.5	7.64×	3.93×

Net carving time, normalized to baseline (lower = faster)

Intel Lunar Lake / iris, interleaved min-of-N, bit-exact. Small tool (32) on long non-axis-aligned segments.

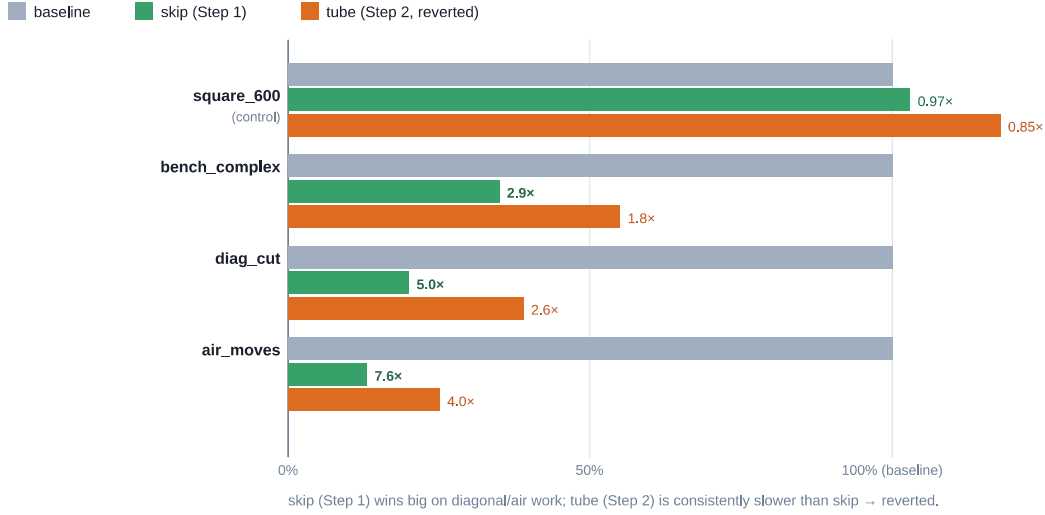


Figure 8. Net carving time normalized to baseline (lower is faster). Tube pruning (skip, Step 1) wins large on diagonal and air-heavy work and is neutral on the axis-aligned control; the oriented tiled dispatch (tube, Step 2) is consistently slower than skip and was reverted.

For historical context, Table 2 recalls how the swept baseline itself was reached from the naive stamper (the pre-tube optimization history on the same class of GPU): baseline stamping ~ 1232 ms $\rightarrow$ per-step overhead removal $\rightarrow$ swept subtraction $\rightarrow$ GPU compaction $\rightarrow$ sub-step bounding ≈ 40 ms total ($\approx 33\times$), all bit-exact. Tube pruning of §3.6 is applied on top of that swept baseline.

6.1 Cross-workload matrix: established methods vs. this work

Table 1 tracks one program; to separate what a *known* method already buys from this work’s contribution, we ran a suite of machining workloads that stress different axes at three refinement levels, all on the **same binary, representation and GPU** — apples-to-apples. We deliberately do **not** quote other authors’ absolute timings (meaningless across hardware, representation, tool and resolution); each level is *our* fair, optimized implementation of the corresponding algorithm class, anchored to the literature it represents: **S0** — **per-step stamping** (the classic z-map/dexel display, Van Hook 1986 [1]); **S1** — **swept-segment subtraction** (the per-move multi-/tri-dexel class, Müller & Surmann 2003 [2]; Tukora & Szalay 2012 [3]; Inui et al. 2019 [4]); **S2** — + **tube pruning** + **in-air segment skip** (this work, §3.6).

Table 3 — carving netto (ms) per machining workload and refinement level
(small hemispheric_mill_3 tool; min of 5 interleaved runs, Intel iris; each level bit-exact within its class):

Workload (what it stresses)	S0 stamping [1]	S1 swept [2–4]	S2 +pruning (ours)	S0→S2	S1→S2
contour — axis-aligned perimeter	45.7	3.90	3.82	12.0×	1.02×
pocket_axis — axis-aligned raster	111.3	7.80	7.87	14.1×	0.99×
raster45 — 45° diagonal raster	187.8	37.1	13.1	14.3×	2.82×
rapids — scattered cuts, long air moves	64.8	11.3	5.26	12.3×	2.15×
localized — small corner feature	34.7	5.36	3.71	9.3×	1.44×
finishing — fine- stepover raster	96.0	6.52	7.75	12.4×	0.84×

S0→S1 ($\approx 9\text{--}14\times$) is the established gain of the per-move swept formulation over per-step stamping — what the known methods already deliver. **S1→S2** is this work’s contribution and is workload-dependent by construction: it removes the over-dispatch of *non-axis-aligned* segments (45° raster **2.82×**) and *in-air* rapids (**2.15×**), is neutral on axis-aligned programs ($\sim 1\times$, the AABB already equals the swept band), and costs a few percent on dense axis-aligned rasters (finishing

0.84×). The mechanism, not an average, is the point. Reproducible via tools/bench_matrix.sh.

Carving time per workload × refinement level (net carving, log scale)

S0 → S1 = established swept method (~12× everywhere); S1 → S2 = this work (tube-pruning + in-air skip). Intel iris, mill_3, min-of-5.

■ S0 stamping [van Hook] ■ S1 swept [Müller&Surmann/tri-dexel] ■ S2 +pruning [this work]

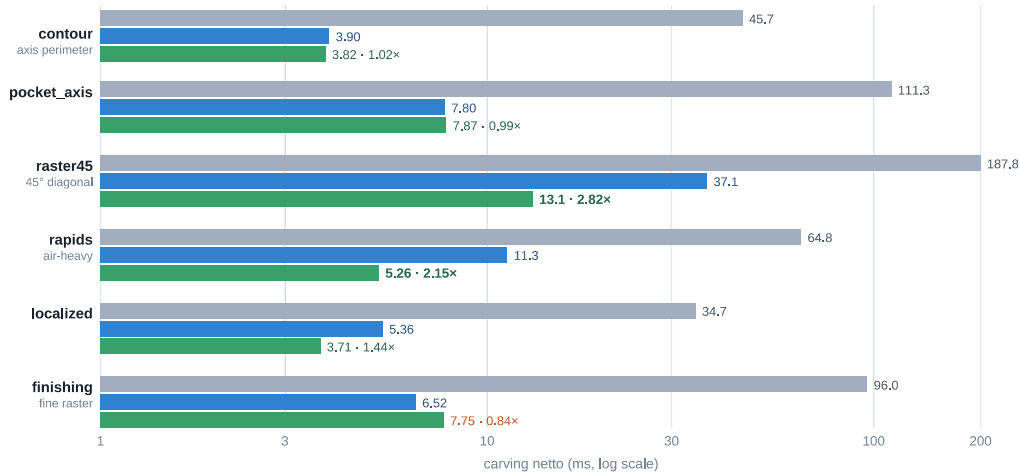


Figure 9. Net carving time per machining workload and refinement level (log scale). S0→S1 (grey→blue) is the established swept method’s uniform ~12× gain over per-step stamping; S1→S2 (blue→green) is this work’s tube-pruning + in-air skip — it pays where the dispatch over-covers (45° raster 2.82×, air-heavy 2.15×), is neutral on axis-aligned work, and costs a little on the dense finishing raster.

6.2 Working-set: a sparse tiled study

The 128 MB flat buffer is ~16× padding (MAX_TRANSITIONS = 32 vs. 2–4 real transitions per column). A full alternative backend (AUTOCAM_CARVE_BACKEND=sparse) stores the stock in 32×32-column tiles, keeping untouched tiles UNIFORM (O(1)) and materializing a tile only when a segment’s bounding box first touches it — byte-identical to the flat backend, runtime-selectable. It isolates two effects. **(i)** Per-carve bandwidth is bounded by the *swept bounding box*, not the buffer size (both backends touch only the bbox columns), so tiling does not reduce the carve’s traffic *volume*; it only improves access *locality* (the strided-access problem) → ~1.1× (square_600 1.09×, bench_complex 1.11×, pocket 1.15×). **(ii)** Footprint is a conditional win tied to locality — a localized program materializes 49–56 / 1024 tiles (**8 MB** vs. 122 MB, ~15×), whereas whole-stock machining touches most tiles (raster45 736 / 1024, no saving). So sparse tiling is a **memory/scale** lever (finer resolutions, larger stocks, smaller read-back for localized work), not a carve-speed lever; host-side materialization even makes whole-stock programs slower.

7. Discussion

The bottleneck is memory-write bandwidth. Three follow-up experiments, read together, localize it:

- *Range-query envelopes (negative)*. Precomputing per-tool directional Z-envelopes as range-minimum/maximum sparse tables makes the per-column envelope an $O(1)$ query instead of an $O(\text{sub-steps})$ loop. Implemented exactly, it gave only ~25–30% — because it optimized *compute*, which is not the bottleneck — and was reverted.
- *Tube pruning (positive, §3.6)*. Skipping the write-back of off-band and out-of-Z-range columns attacks *memory-write traffic* and gives 2.6–7.6× in its regime. This is the complement of the previous result: the lever that pays removes memory traffic, not compute.
- *Oriented tiled dispatch (negative)*. Tempted to also avoid *launching* the off-band threads (each still does a counter read plus the early-out), we tiled the dispatch along the motion so it follows the band. Each tile reuses the full-segment envelope uniforms (so every column is computed identically); adjacent tiles overlap idempotently, separated by a barrier — keeping it bit-exact. The result was ~2× **slower** than tube pruning alone: the per-tile barriers serialize the tiles and the joins double-process columns, and that synchronization cost exceeds the thread-launch saving, because tube pruning already made the off-band threads cheap (a read, no write). Reverted.
- *Sparse tiled working-set (footprint, not speed; §6.2)*. A tiled backend that keeps untouched tiles UNIFORM shrinks the working set to the touched fraction, but the per-carve traffic is bounded by the swept bounding box in either layout, so it only improves access *locality* (~1.1×) and reduces *footprint* conditionally on locality (~15× localized, nothing whole-stock). Working-set size is a memory/scale lever, not a bandwidth one.

The pattern is consistent — a **map of the bottleneck**. Attacking *compute* (RMQ), *launched-thread count* (tiled dispatch) or *working-set size* (sparse tiling) yields diminishing, negative or footprint-only returns; attacking *memory-write traffic on the over-dispatched bounding box* (tube pruning + in-air skip) is the only lever that breaks the ceiling. The simulator is **bbox-bandwidth-bound**: the productive optimization is the one that removes bbox traffic. A dispatch-enqueue cost that

stays ~0.4 ms whether a program has 9 or 202 segments corroborates that per-segment host overhead is not the limit.

When tube pruning helps. The gain requires segment length $L \gg D$. Axis-aligned programs, or diagonal programs machined with a tool nearly as wide as the segments are long, see little — their AABB already equals the tube. Real 3-axis programs, however, contain substantial air travel (rapids between features, retract/reposition links) and non-axis-aligned finishing passes, all of which this optimization accelerates; the in-air pruning in particular applies to **every** rapid.

Limitations. The pipeline runs in voxel units; a calibrated millimetre mapping is future work. The envelope over-removes for non-convex-in-Z tools, and the flat-buffer stride `MAX_TRANSITIONS = 32` is not overflow-guarded. The tool is axis-fixed (3-axis only). The largest remaining lever for the optimization loop is a **GPU-side objective** (comparing carved-vs-target on the GPU) to remove the per-iteration read-back entirely.

8. Conclusions

We presented a GPU material-removal simulator for 3-axis milling built on a single-axis dixel field, in which Boolean difference is a per-column merge that maps one workpiece column to one GPU thread. On top of a fused, materialization-free swept-segment subtraction, we introduced **tube pruning** — skipping the write-back for columns the swept tool never touches — which is bit-exact and yields 2.6–7.6× on the diagonal and air-heavy work that dominates real toolpaths, while leaving axis-aligned programs unchanged. The optimization, and two documented negative results bracketing it, together show that on this pipeline the decisive resource is **memory-write bandwidth**: the successful lever removes bytes written, not arithmetic or launched threads. The simulator is fast enough and accurate enough (bit-exact) to serve as the forward model of a search- or learning-based toolpath generator, which is the subject of ongoing work.

Appendix A. Terminology

Term	Meaning in this paper
Voxel	A cell of the regular 3-D grid; solid or empty.

Term	Meaning in this paper
Dexel	“Depth element”: the run of solid voxels along one axis in a column.
Single-axis dexel field	The stock/tool stored as per-(x,y)-column Z-runs (our representation).
Column	One (x, y) stack of D voxels along Z.
Transition	A Z height where a column’s occupancy flips (empty ↔ solid).
Parity walk	Recovering occupancy by counting transitions $\leq z$: odd $\Rightarrow$ solid.
Stock / workpiece	The block being machined (obj1).
Tool	The cutter (obj2); here a ball-end mill (hemisphere + shank).
Footprint (D)	The tool’s XY extent in voxels ($w_2 \times h_2$).
Segment	One straight toolpath move $p_0 \rightarrow p_1$.
Swept volume	The region the tool sweeps over a segment ($\text{tool} \oplus p_0p_1$).
Z-envelope	Per stock column, the single interval [min bottom, max top] of the swept tool.
Sub-step	One sampled tool-centre position along a segment.
Chebyshev length (K)	$\max(\Delta x , \Delta y , \Delta z)$; number of ≤ 1 -voxel sub-steps.
AABB	Axis-aligned bounding box (of a segment’s endpoints, dilated by the footprint).
Swept band / tube	The columns the tool actually sweeps — a band along the motion.
Tube pruning	Skipping the write-back for columns outside the tube / stock Z-range.
Thread	One GPU work-item; here it processes one column.
Workgroup	A block of threads executed together (8×8 here).
Dispatch	A launch of a compute shader over a

Term	Meaning in this paper
	grid of workgroups.
gl_GlobalInvocationID	A thread's global index within the dispatch grid.
SSBO	Shader Storage Buffer Object: a large GPU-resident array.
Memory barrier	A synchronization that orders dependent GPU memory accesses.
MAX_TRANSITIONS	Fixed per-column slot count (32) of the flat working buffer.
Bit-exact	Byte-identical output; verified via the residual-volume invariant.
Residual solid volume	Σ (top – bottom) over all columns' solid intervals.
Net carving time	GPU carving-loop time, excluding read-back (measured with a glFinish).

References

1. T. Van Hook. *Real-time shaded NC milling display*. Computer Graphics (Proc. SIGGRAPH '86), 20(4):15–20, 1986.
2. H. Müller, T. Surmann, M. Stautner, F. Albersmann, K. Weinert. *Online sculpting and visualization of multi-dexel volumes*. Proc. 8th ACM Symposium on Solid Modeling and Applications (SM '03), 258–261, 2003.
3. B. Tukora, T. Szalay. *Multi-dexel based material removal simulation and cutting force prediction with the use of general-purpose graphics processing units*. Advances in Engineering Software, 43(1):65–70, 2012.
4. M. Inui, M. Kobayashi, N. Umezu. *Cutter engagement feature extraction using triple-dexel representation workpiece model and GPU parallel processing*. Computer-Aided Design and Applications, 16(1):89–102, 2019.
5. C. C. L. Wang, Y.-S. Leung, Y. Chen. *Solid modeling of polyhedral objects by Layered Depth-Normal Images on the GPU*. Computer-Aided Design, 42(6):535–544, 2010.
6. D. Blackmore, M. C. Leu, L. P. Wang. *The sweep-envelope differential equation algorithm and its application to NC machining verification*. Computer-Aided Design, 29(9):629–637, 1997.

7. K. Abdel-Malek, J. Yang, D. Blackmore, K. Joy. *Swept volumes: foundation, perspectives, and applications*. International Journal of Shape Modeling, 12(1):87–127, 2006.
8. W. E. Lorensen, H. E. Cline. *Marching cubes: a high resolution 3D surface construction algorithm*. Computer Graphics (Proc. SIGGRAPH '87), 21(4):163–169, 1987.
9. C. Dyken, G. Ziegler, C. Theobalt, H.-P. Seidel. *High-speed marching cubes using HistoPyramids*. Computer Graphics Forum, 27(8):2028–2039, 2008.